

Robotik Kazıma İçin Takım Yolu Oluşturma

Tool Path Generation For Robotic Milling

Murat Can YİĞİTOĞLU¹,

¹ Elektronik ve Haberleşme Mühendisliği, Kocaeli Üniversitesi, Kocaeli, 41100, Türkiye, **ORCID:** 0009-0005-3482-3624

Makale Bilgisi	Özet
Proje Raporu	Robotik kazıma teknolojisi, robotların esnek yapısı sayesinde geleneksel CNC yöntemlere nazaran alternatif bir üretim yolu olarak görülmektedir. ABB IRB 1600 robot kolu ve matkap kullanılarak bir sistem geliştirilmiştir. Çalışma kapsamında, işlenecek görsel verilerin robot yörüngesine dönüştürülmesi amacıyla Python tabanlı bir algoritma geliştirilmiştir. Bu algoritma ile görsel veriler, ABB IRB 1600 robotunun tanıyabileceği .mod ve .txt formatındaki komut dizilerine dönüştürülerek Robot Studio ortamında simüle edilmiştir.
Anahtar Kelimeler	
Robotik Kazıma, Robot Studio, Python, Robot Yazılımı	
Article Info	Abstract
Project Report	Robotic milling technology is seen as an alternative production method to traditional CNC methods due to the flexible nature of robots. A system has been developed using an ABB IRB 1600 robotic arm and drill. Within the scope of this study, a Python-based algorithm was developed to transform visual data into robot trajectories. By utilizing this algorithm, visual data were converted into command sequences in .mod and .txt formats compatible with the ABB IRB 1600 robot, and subsequently simulated within the RobotStudio environment.
Keywords	
Robotic Milling, Robot Studio, Python, Robot Ware	

1.Giriş

Endüstriyel robotların üretim süreçlerinde esnek bir şekilde kullanılabilmesi, karmaşık tasarım verilerinin robot hareket komutlarına hatasız bir şekilde dönüştürülmesine bağlıdır. Geleneksel üretim yöntemlerinde yaygın olarak kullanılan G-Code mantığı, S. B. Niku'nun [1] numaralı yayınında belirttiği robotik sistem yaklaşımıyla uyumlu şekilde, robotun eklem yapısına ve çalışma uzayına uygun koordinat verileriyle yönetilmektedir. Bu çalışma kapsamında, bir görselin veya tasarımın dijital ortamdan alınarak ABB IRB 1600 tipi bir robot kolunun takip edebileceği noktasal veri kümelerine dönüştürülmesi süreci ele alınmıştır. Projenin temel teknik odağını; ham görsel

verisinin analiz edilmesi, hassas XYZ koordinat bilgilerinin üretilmesi ve bu bilgilerin sistematik bir işleme yoluna dönüştürülmesi oluşturmaktadır. J. Norberto Pires'in [2] numaralı çalışmasında detaylandığı endüstriyel robot programlama prensipleri doğrultusunda, geliştirme sürecinde görüntü dosyaları Python tabanlı algoritmalar [3] ile işlenmiş ve yüzey üzerindeki her bir noktanın robotun çalışma uzayındaki gerçek dünya koordinatlarına eşlenmesi sağlanmıştır. Üretilen bu noktasal veri setleri sadece koordinat çıkarımı değil, aynı zamanda robotun izleyeceği yolun optimizasyonunu da kapsamaktadır. Hazırlanan bu veri setlerinin doğruluğu, [4] numaralı kaynakta belirtilen ABB RobotStudio operasyonel standartları ve simülasyon metodolojisi kullanılarak test edilmiş, üretilen XYZ

çıktılarının sanal ortamdaki başarısı teyit edilmiştir.

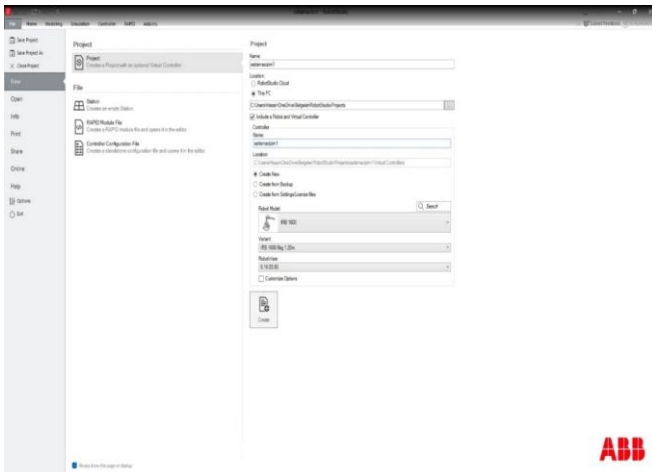
2. Malzeme ve Yöntem

Bu çalışmanın geliştirme sürecinde fiziksel bir üretim materyali yerine tamamen dijital tasarım ve simülasyon araçlarından faydalanılmıştır. Çalışmanın ana materyalleri arasında vektörel çizimlerin oluşturulması koordinat verilerinin işlenmesi ve robotik simülasyonun gerçekleştirilmesi için kullanılan yazılım paketleri yer almaktadır. İlk aşamada işlenecek olan görsel verilerin düzenlenmesi ve uygun formatta dışa aktarılması amacıyla Inkscape yazılımı kullanılmıştır. Inkscape üzerinden elde edilen vektörel veriler projenin temelini oluşturan sayısal koordinatların türetilmesi için bir alt yapı sağlamıştır. İkinci aşamada ise bu verilerin robotun anlayabileceği XYZ formatındaki noktasal veri setlerine dönüştürülmesi süreci

3. Tasarım

3.1. Robot ve Donanım Seçimi

Projenin temel taşı olan robot modelinin belirlenmesinde; çalışma alanının genişliği, hassasiyet kriterleri ve yazılımsal entegrasyon kabiliyeti öncelikli olarak göz önünde bulundurulmuştur. Bu doğrultuda seçilen ABB IRB 1600-8/1.45 modeli, 1.45 metrelik uzun erişim kapasitesi sayesinde Saitama çiziminin gerektirdiği geniş hareket alanını güvenli bir şekilde sağlamaktadır. Robotun 8 kilogramlık taşıma kapasitesi, uç birime eklenecek olan kalem tutucu aparatın mekanik gereksinimlerini fazlasıyla karşılamaktadır. Sistemin yazılımsal kontrolü ve sanal devreye alma süreci için RobotWare 6 işletim sistemi versiyonuna sahip bir sanal kontrolcü yapılandırılmış; bu sayede robotun tüm eklem hareketleri ve çalışma uzayı **Şekil 1.**'de gösterilen simülasyon ortamında gerçeğe en yakın şekilde test edilmiştir..



Şekil 1. Robot Studio Simülasyon Kurulumu.

Python programlama dili ve ilgili kütüphaneler aracılığıyla gerçekleştirilmiştir. Python tabanlı algoritmalar kullanılarak görsel üzerindeki her bir hat sistematik bir şekilde analiz edilmiş ve bu hatlar robotun çalışma uzayına uygun kartezyen koordinat dizilerine dönüştürülmüştür. Yöntem olarak belirlenen bu akış sayesinde dijital bir tasarımın G-code mantığına benzer bir yapılandırma ile robotik işleme yoluna (toolpath) evrilmesi sağlanmıştır. Üretilen bu koordinat listelerinin ve yol planlamasının doğrulanması için son aşamada ABB RobotStudio simülasyon yazılımı kullanılmıştır. Simülasyon sürecinde ABB IRB 1600 endüstriyel robot kolu sanal ortamda modellenmiş ve Python üzerinden üretilen XYZ verileri bu robotun hareket komutlarına entegre edilmiştir. Bu yöntemle koordinatların hassasiyeti ve robotun çalışma alanı içerisindeki hareket sınırları fiziksel bir kurulumla gerek duyulmadan dijital ortamda test edilerek doğrulanmıştır.

3.2. Yazılım Mimarisi ve Görüntü İşleme Süreci

Projenin temelini, dijital bir görseli analiz ederek robotun anlayabileceği RAPID komutlarına dönüştüren Python tabanlı bir yazılım oluşturmaktadır. Yazılımın işleyişi, kodun fonksiyonel bloklarına göre aşağıda detaylandırılmıştır:

3.2.1. Kütüphane Yapılandırması ve Dosya Yönetimi

Yazılımın ilk aşamasında, görüntü işleme için OpenCV (cv2), matris tabanlı veri işlemleri için numpy ve dosya sistemine erişim için os kütüphaneleri dahil edilmiştir. Bu bölümde ayrıca giriş yapılacak görselin adı ve çıktı olarak üretilcek robot program dosyasının (.mod) isim tanımlamaları yapılmaktadır.

```
import cv2
import numpy as np
import os

resim_adi = 'saitama.png'
cikis_dosyasi = 'SaitamaRobot.mod'
```

Şekil 2: Yazılım Kütüphane ve Dosya Tanımlamaları.

3.2.2. Veri Okuma ve Hata Denetim Mekanizması

Yazılımın güvenli çalışması için try-except blokları kullanılarak dosya okuma kontrolleri gerçekleştirilir. os.path.exists kontrolü ile resim dosyasının varlığı teyit edilir. cv2.imdecode fonksiyonu ile görsel verisi bellek üzerine alınarak işleme hazır hale getirilir.

```
def rapid_olustur():
    resim_yolu=os.path.join(current_dir,
    resim_adi)
    if not os.path.exists(resim_yolu):
        print(f'HATA: '{resim_adi}' resmi
        bulunamadı!')
    return
```

Şekil 3: Veri Okuma ve Hata Kontrol Algoritması.

3.2.3. Görüntü Ön İşleme ve Kontur Analizi

Okunan görsel, analiz kolaylığı için gri seviyeye dönüştürülür ve GaussianBlur filtresi ile dijital gürültülerden arındırılır. Ardından Canny algoritması ile nesne kenarları (pikselleri) tespit edilir. Tespit edilen bu pikseller, findContours fonksiyonu ile robotun takip edebileceği sürekli yörünge listelerine (contours) dönüştürülür. Bu aşamadaki yazılım çıktısı Şekil 2’de görselleştirilmiştir.

```
gray=cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 50, 150)
contours,_=cv2.findContours(edges,
cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
```

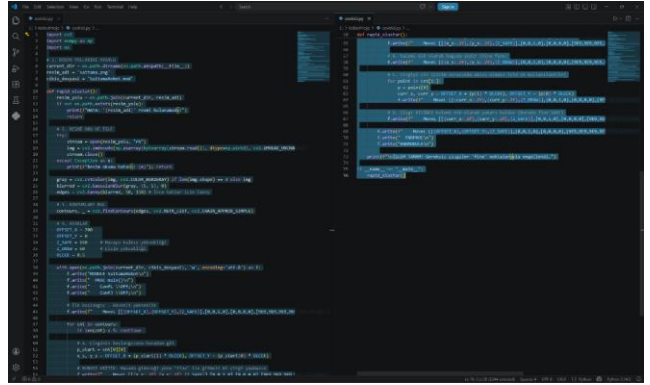
Şekil 4: OpenCV Kenar ve Kontur Analizi Kod Bloğu.

3.2.4. Robotik Koordinat Haritalama ve Ofset Parametreleri

Piksel koordinatlarının gerçek dünyaya (milimetre) aktarılması için matematiksel bir modelleme yapılmıştır. OFFSET_X ve OFFSET_Y değerleri çizimin robotun çalışma alanındaki merkezini belirlerken, OLCEK parametresi resmin boyutlarını robotun erişim sınırlarına uyarlamaktadır. Z_SAFE (150mm) ve Z_DRAW (50mm) değişkenleri ise kalem uç biriminin dikey hareket limitlerini belirleyen kritik yazılım parametreleridir.

```
OFFSET_X = 700
OFFSET_Y = 0
Z_SAFE = 150      # Havaya kalkış yüksekliği
Z_DRAW = 50      # Çizim yüksekliği
OLCEK = 0.5      # Ölçeklendirme faktörü
```

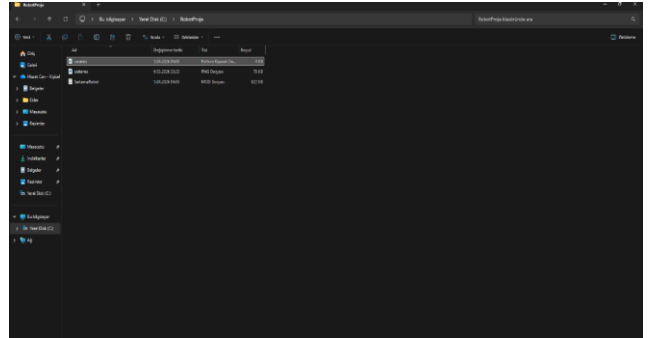
Şekil 5: Koordinat Ölçekleme ve Z Eksen Parametreleri.



Şekil 6: Python Yazılımı Tam Kod Dizilimi.

3.2.5. Proje Dosya Yapısı ve Çıktı Yönetimi

Yazılımın çalışma prensibi doğrultusunda, girdi dosyası olan görsel ve Python betiği aynı dizin içerisinde işlenmektedir. Algoritmanın başarılı bir şekilde koşturulması sonucunda, robot kontrolcüsüne aktarılmaya hazır olan .mod uzantılı program dosyası otomatik olarak oluşturulmaktadır. Yazılımın dosya hiyerarşisi ve oluşan çıktı dosyası Şekil 7’de gösterilmiştir.



Şekil 7: Yazılım Giriş-Çıkış Dosyaları ve Dizin Yapısı.

3.3. Koordinat Dönüşümü ve Hareket Planlama

Görüntü işleme aşamasında elde edilen veriler, Python algoritmaları aracılığıyla 10.655 satırlık bir RAPID (robot programlama dili) dosyasına dönüştürülmüştür. Bu bölümde, üretilen kodun yapısı ve robotun bu verileri nasıl fiziksel harekete döndürdüğü analiz edilmektedir.

3.3.1. RAPID Hareket Komutları ve Veri Analizi

Yazılım, her bir kontur noktasını ABB robotlarının standart doğrusal hareket komutu olan MoveL formatına çevirir. Üretilen program dosyası incelendiğinde, her hareket satırının beş temel bileşenden oluştuğu görülmektedir:

Dosyadaki Örnek Satır: > MoveL [[754.20, 25.40, 50.00], [0,0,1,0], [0,0,0,0], [9E9...]], v150, z1, tool0;

1. Hedef Koordinat (754.20, 25.40, 50.00): Robotun milimetrik düzlemdeki X, Y ve Z konumudur. Burada 50.00 mm değeri, kalemin kağıda temas ettiği hassas çizim yüksekliğini (Z_DRAW) temsil eder.

2.Yönelim (0,0,1,0): Kalem ucunun kağıda her zaman dik (z-ekseni doğrultusunda) kalmasını sağlayan kuaternyon değeridir.

3.Hız Verisi (v150): Çizim kalitesini korumak için robotun saniyede 150 mm hızla ilerlediğini gösterir.

4.Hassasiyet (z1): Çizim sırasında robotun duraksamaması için 1 mm'lik geçiş toleransdır.

5.Takım Verisi (tool0): Hareketin robotun standart uç birim merkezine göre hesaplandığını belirtir.

3.3.2. Çizim Kalitesi: Fine ve Zone Kontrolü

Kod içerisinde uygulanan "Termination Data" mantığı, çizimin netliğini belirleyen en kritik yazılımsal unsurdur. Dosya içeriğinde iki farklı strateji izlenmiştir:

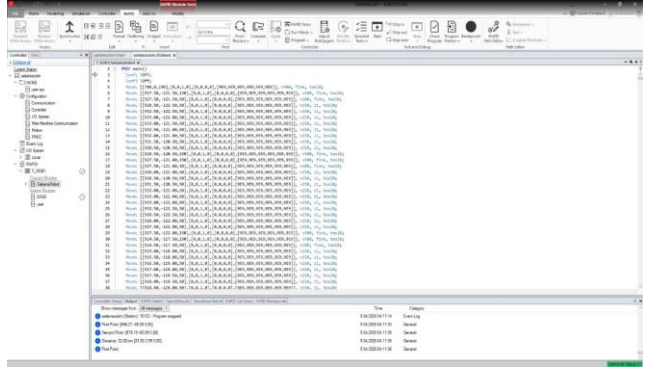
- **fine:** Çizginin başladığı ilk noktada ve bittiği son noktada robotun tam duruş yapması sağlanmıştır. Bu, çizgilerin birbirine karışmasını ve köşelerin yuvarlanmasını engeller.
- **z1:** Sürekli çizgiler (konturlar) boyunca robotun akıcı bir şekilde ilerlemesi sağlanarak sarsıntısız bir çizim elde edilmiştir.

3.3.3. Güvenli Geçiş ve Havaya Kalkış Algoritması

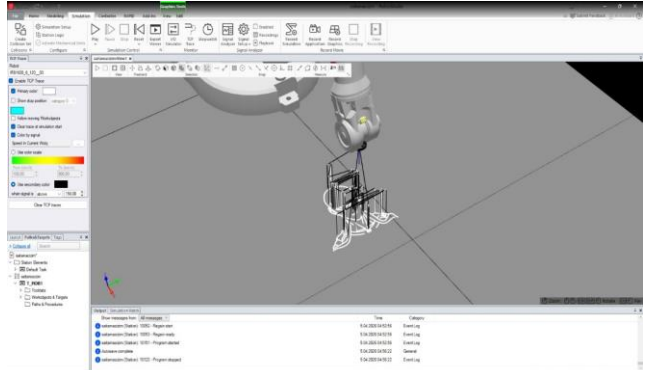
Robotun bir çizgiyi bitirip diğerine geçerken kağıdı çizmemesi ("hayalet çizgi" oluşmaması) için yazılımda bir dikey hareket mantığı kurulmuştur. Kod yapısında görüldüğü üzere, her kontur bitiminde robot Z_SAFE (150 mm) yüksekliğine çıkarılmakta, ardından yeni çizgi başlangıcına havadan gitmektedir. Bu süreçte kullanılan v500 hızı, çizim dışı geçişlerin hızlı tamamlanmasını sağlayarak işlem süresini optimize etmektedir. Üretilen bu 10.000 satırı aşkın yörünge yapısı Şekil 8'de gösterilmiştir.

3.3.4. Üretilen Program Dosyası ve Simülasyon Çıktısı

Hazırlanan Python algoritması sonucunda oluşan 10.000 satırı aşkın hareket komutu, robotun hafızasına aktarılmaya hazır hale getirilmiştir. Yazılımın ürettiği ham koordinat verileri ve bu verilerin RobotStudio ortamındaki fiziksel izdüşümü sırasıyla Şekil 8 ve Şekil 9'da sunulmuştur.



Şekil 8: Üretilen RAPID Programı ve Örnek MoveL Koordinat Dizilimleri.



Şekil 9: 10.655 Satırlık Kodun Simülasyon Ortamındaki Nihai Çizim Sonucu.

4. Sonuçlar

Bu çalışma ile dijital bir görselin endüstriyel bir robot yörüngesine dönüştürülmesi süreci başarıyla tamamlanmıştır. Elde edilen sonuçlar projenin teknik hedefleri doğrultusunda şu şekilde özetlenmiştir:

4.1. Sistemin Çalışma Başarımı

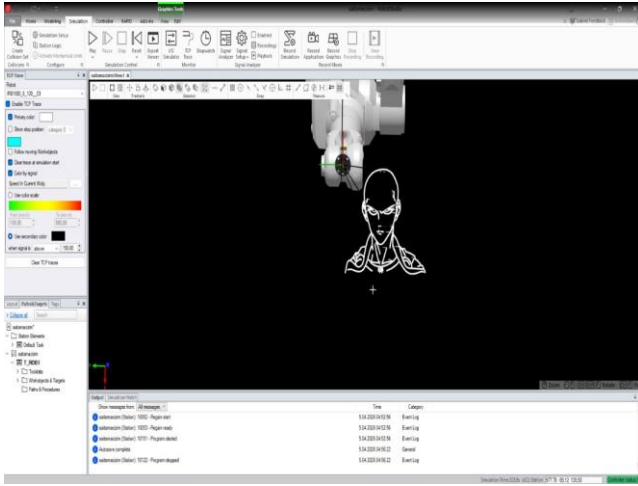
Geliştirilen Python tabanlı yazılım, Saitama görselini 10.655 satırlık hatasız bir RAPID koduna dönüştürmüştür. Canny Edge Detection algoritması sayesinde karakterin tüm karakteristik detayları korunmuş, robotun sarsıntısız bir şekilde takip edebileceği pürüzsüz bir yörünge seti elde edilmiştir.

4.2. Simülasyon Verimliliği ve Çizim Kalitesi

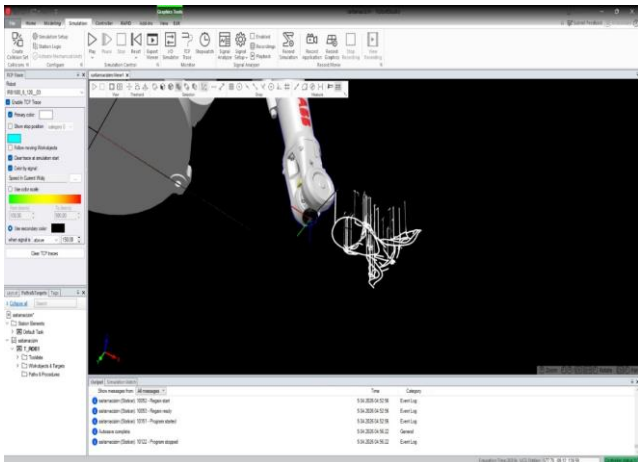
RobotStudio ortamında yapılan testlerde, yazılımda tanımlanan Z_SAFE (havaya kalkış) mantığının kusursuz çalıştığı ve çizgiler arası geçişlerde oluşan "hayalet çizgilerin" tamamen engellendiği doğrulanmıştır. fine ve z1 hassasiyet parametrelerinin dinamik yönetimi sayesinde, hem keskin hatlar net bir şekilde yakalanmış hem de uzun konturlar akıcı bir şekilde çizilmiştir.

4.3. Nihai Değerlendirme

Proje sonunda, görüntü işleme ve robotik programlamanın entegre edildiği uçtan uca bir sistem geliştirilmiştir. Sistemin ürettiği nihai simülasyon çıktıları ve robotun yörünge üzerindeki çizim performansı Şekil 10 ve Şekil 11'de sunulmuştur.



Şekil 10: Robotun Yörünge Üzerindeki Aktif Çizim Süreci.



Şekil 11: Tamamlanan 10.655 Sıralık Yörünge Planının Nihai Simülasyon Çıktısı.

Etik Standartlar Beyanı

Bu makalenin yazarı, bu çalışmada kullanılan materyal ve yöntemlerin etik kurul izni ve / veya yasal-özel izin gerektirmediğini beyan etmektedir.

Çıkar Çatışması

Yazar, bu makalede bildirilen çalışmayı etkilemiş gibi görünebilecek, bilinen rakip mali çıkarları veya kişisel ilişkileri olmadığını beyan eder.

Kaynaklar

- [1] S. B. Niku, *Introduction to Robotics: Analysis, Control, Applications*, 3rd ed., Wiley, 2019.
- [2] J. Norberto Pires, *Industrial Robots Programming: Building Applications for the Factories of the Future*, Springer, 2007.
- [3] Python Software Foundation, "Python Language Reference, version 3.x," [Online]. Available: <https://www.python.org/> [Accessed: 04-Apr-2026].
- [4] ABB Robotics, "RobotStudio Operating Manual," [Online]. Available: <https://new.abb.com/products/robotics/robotstudio> [Accessed: 04-Apr-2026].